

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра информационных технологий и автоматизированных систем

ОТЧЕТ
о лабораторной работе 1
Семестр: 4

На тему: «Матрица достижимости и компоненты связности графа»

Выполнил студент ИВТ-24-26:

Шишкин М.Г

(дата, подпись)

Проверил:

Рустамханова Г.И

(дата, подпись)

Пермь 2026

Содержание

1 Постановка задачи	3
2 Краткое описание решения	4
3 Код	5
4 Тестирование программы	12
5 Вывод	15
6 Github	16

1 Постановка задачи

Разработать программу на C++ для анализа графа по матрице смежности 10×10 из файла. Программа определяет тип графа (ориентированный / неориентированный). Если граф неориентированный, матрица приводится к симметричному виду. Находятся компоненты связности, выполняются обходы в глубину (DFS) и ширину (BFS). Строится матрица достижимости через объединение степеней матрицы до стабилизации. Также строится матрица компонент сильной связности.

2 Краткое описание решения

Программа на C++ последовательно:

1. Считывает матрицу из файла.
2. Проверяет симметричность: если $\text{matrix}[i][j] \neq \text{matrix}[j][i]$ — граф ориентированный, иначе неориентированный.
3. Для неориентированного графа делает матрицу симметричной (доопределяет до ориентированного).
4. Находит компоненты связности рекурсивным DFS.
5. Выполняет обходы: итеративный DFS (стек) и BFS (очередь).
6. Строит матрицу достижимости: последовательно перемножает матрицы (булево умножение) и объединяет их (логическое ИЛИ). Алгоритм останавливается при совпадении матриц на двух последних шагах. На главную диагональ добавляются единицы.
7. Строит матрицу компонент сильной связности: единица на позициях (i, j) , если $\text{reachMatrix}[i][j] == 1$ и $\text{reachMatrix}[j][i] == 1$.

3 Код

```
1  #include <iostream>
2  #include <fstream>
3  #include <vector>
4  #include <queue>
5  #include <stack>
6  #include <algorithm>
7  #include <sstream>
8
9  using namespace std;
10
11  const int N = 10; // Размер матрицы
12
13  // Функция для чтения матрицы из файла
14  vector<vector<int>> readMatrix(const string& filename) {
15      ifstream file(filename);
16      vector<vector<int>> matrix(N, vector<int>(N, 0));
17
18      if (!file.is_open()) {
19          cerr << "Не удалось открыть файл!" << endl;
20          return matrix;
21      }
22
23      for (int i = 0; i < N; i++) {
24          for (int j = 0; j < N; j++) {
25              file >> matrix[i][j];
26          }
27      }
28
29      file.close();
30      return matrix;
31  }
32
33  // Функция для вывода матрицы
34  void printMatrix(const vector<vector<int>>& matrix, const string& name) {
35      cout << "\n" << name << ":\n";
36      for (int i = 0; i < N; i++) {
37          for (int j = 0; j < N; j++) {
38              cout << matrix[i][j] << " ";
39          }
40          cout << endl;
41      }
42  }
43
44  // Перегрузка для вывода без названия (для обратной совместимости)
45  void printMatrix(const vector<vector<int>>& matrix) {
46      for (int i = 0; i < N; i++) {
47          for (int j = 0; j < N; j++) {
48              cout << matrix[i][j] << " ";
49          }
50          cout << endl;
```

```

52 }
53
54 // Проверка на ориентированность (если есть хоть одна пара (i,j) != (j,i), то ориентированный)
55 bool isOriented(const vector<vector<int>>& matrix) {
56     for (int i = 0; i < N; i++) {
57         for (int j = 0; j < N; j++) {
58             if (matrix[i][j] != matrix[j][i]) {
59                 return true; // Есть несимметричность -> ориентированный
60             }
61         }
62     }
63     return false; // Полностью симметричный -> неориентированный
64 }
65
66 // Преобразование в ориентированный (делаем симметричным)
67 vector<vector<int>> makeOriented(const vector<vector<int>>& matrix) {
68     vector<vector<int>> newMatrix = matrix;
69     for (int i = 0; i < N; i++) {
70         for (int j = 0; j < N; j++) {
71             if (matrix[i][j] == 1 || matrix[j][i] == 1) {
72                 newMatrix[i][j] = 1;
73                 newMatrix[j][i] = 1;
74             }
75         }
76     }
77     return newMatrix;
78 }
79
80 // Обход в глубину (DFS) для поиска компонент связности
81 void dfs(int v, vector<bool>& visited, const vector<vector<int>>& matrix, vector<int>& component) {
82     visited[v] = true;
83     component.push_back(v + 1); // Запоминаем вершину (нумерация с 1 для вывода)
84
85     for (int u = 0; u < N; u++) {
86         if (matrix[v][u] == 1 && !visited[u]) {
87             dfs(u, visited, matrix, component);
88         }
89     }
90 }
91
92 // Поиск всех компонент связности
93 vector<vector<int>> findComponents(const vector<vector<int>>& matrix) {
94     vector<bool> visited(N, false);
95     vector<vector<int>> components;
96
97     for (int i = 0; i < N; i++) {
98         if (!visited[i]) {
99             vector<int> component;
100             dfs(i, visited, matrix, component);
101             components.push_back(component);

```

```

102     }
103 }
104
105     return components;
106 }
107
108 // Обход в ширину (BFS) из заданной вершины
109 void bfs(int start, const vector<vector<int>>& matrix) {
110     vector<bool> visited(N, false);
111     queue<int> q;
112     vector<int> traversal;
113
114     visited[start] = true;
115     q.push(start);
116
117     while (!q.empty()) {
118         int v = q.front();
119         q.pop();
120         traversal.push_back(v + 1);
121
122         for (int u = 0; u < N; u++) {
123             if (matrix[v][u] == 1 && !visited[u]) {
124                 visited[u] = true;
125                 q.push(u);
126             }
127         }
128     }
129
130     cout << "BFS из вершины " << start + 1 << ": ";
131     for (int v : traversal) {
132         cout << v << " ";
133     }
134     cout << endl;
135 }
136
137 // Обход в глубину (DFS) из заданной вершины
138 void dfsTraversal(int start, const vector<vector<int>>& matrix) {
139     vector<bool> visited(N, false);
140     stack<int> st;
141     vector<int> traversal;
142
143     st.push(start);
144
145     while (!st.empty()) {
146         int v = st.top();
147         st.pop();
148
149         if (!visited[v]) {
150             visited[v] = true;
151             traversal.push_back(v + 1);

```

```

151 traversal.push_back(v + 1);
152
153 // Идем в обратном порядке, чтобы сохранить порядок как в рекурсивном DFS
154 for (int u = N - 1; u >= 0; u--) {
155     if (matrix[v][u] == 1 && !visited[u]) {
156         st.push(u);
157     }
158 }
159 }
160
161 cout << "DFS из вершины " << start + 1 << ": ";
162 for (int v : traversal) {
163     cout << v << " ";
164 }
165 cout << endl;
166 }
167
168 // Умножение булевых матриц (объединение)
169 vector<vector<int>> multiplyMatrix(const vector<vector<int>>& a, const vector<vector<int>>& b) {
170     vector<vector<int>> result(N, vector<int>(N, 0));
171
172     for (int i = 0; i < N; i++) {
173         for (int j = 0; j < N; j++) {
174             for (int k = 0; k < N; k++) {
175                 if (a[i][k] && b[k][j]) {
176                     result[i][j] = 1;
177                     break;
178                 }
179             }
180         }
181     }
182
183     return result;
184 }
185
186 // Объединение матриц (поэлементное ИЛИ)
187 vector<vector<int>> unionMatrix(const vector<vector<int>>& a, const vector<vector<int>>& b) {
188     vector<vector<int>> result(N, vector<int>(N, 0));
189
190     for (int i = 0; i < N; i++) {
191         for (int j = 0; j < N; j++) {
192             result[i][j] = a[i][j] || b[i][j];
193         }
194     }
195
196     return result;
197 }
198
199 // Проверка равенства матриц
200

```



```

201 bool isEqual(const vector<vector<int>>& a, const vector<vector<int>>& b) {
202     for (int i = 0; i < N; i++) {
203         for (int j = 0; j < N; j++) {
204             if (a[i][j] != b[i][j]) return false;
205         }
206     }
207     return true;
208 }
209
210 // Функция для преобразования числа в строку (замена to_string)
211 string intToString(int num) {
212     stringstream ss;
213     ss << num;
214     return ss.str();
215 }
216
217 // Построение матрицы достижимости (транзитивное замыкание)
218 vector<vector<int>> reachabilityMatrix(vector<vector<int>> matrix) {
219     vector<vector<int>> prev = matrix;
220     vector<vector<int>> current = matrix;
221     vector<vector<int>> result = matrix;
222
223     int step = 1;
224     cout << "\nПостроение матрицы достижимости:\n";
225     cout << "\nWar 0 (A^1):\n";
226     printMatrix(result);
227
228     while (true) {
229         step++;
230         current = multiplyMatrix(current, matrix); // A^k = A^(k-1) * A
231         result = unionMatrix(result, current);
232
233         cout << "\nWar " << step - 1 << " (объединение до A^" << step << "):\n";
234         printMatrix(result);
235
236         if (isEqual(result, prev)) {
237             break;
238         }
239
240         prev = result;
241     }
242
243     // Добавляем единичную матрицу (достижимость самой себя)
244     for (int i = 0; i < N; i++) {
245         result[i][i] = 1;
246     }
247
248     return result;
249 }

```

```

249     }
250
251     // Матрица компонент сильной связности
252     vector<vector<int>> strongComponentsMatrix(const vector<vector<int>>& reachMatrix) {
253         vector<vector<int>> compMatrix(N, vector<int>(N, 0));
254
255         for (int i = 0; i < N; i++) {
256             for (int j = 0; j < N; j++) {
257                 // Вершины в одной компоненте сильной связности, если достижимы друг из друга
258                 if (reachMatrix[i][j] == 1 && reachMatrix[j][i] == 1) {
259                     compMatrix[i][j] = 1;
260                 }
261             }
262         }
263
264         return compMatrix;
265     }
266
267     int main() {
268         setlocale(LC_ALL, "Russian");
269
270         // 1. Чтение матрицы из файла
271         string filename = "g14.txt";
272         vector<vector<int>> matrix = readMatrix(filename);
273
274         cout << "Исходная матрица смежности:";
275         printMatrix(matrix, "");
276
277         // 2. Проверка на ориентированность
278         bool oriented = isOriented(matrix);
279         cout << "\nГраф " << (oriented ? "ориентированный" : "неориентированный") << endl;
280
281         // 3. Если неориентированный - делаем ориентированным (симметричным)
282         vector<vector<int>> workMatrix;
283         if (!oriented) {
284             cout << "Преобразуем в ориентированный (делаем симметричным)..." << endl;
285             workMatrix = makeOriented(matrix);
286             printMatrix(workMatrix, "Ориентированная матрица");
287         }
288         else {
289             workMatrix = matrix;
290         }
291
292         // 4. Компоненты связности (для неориентированного графа)
293         cout << "\n=== Компоненты связности ===" << endl;
294         vector<vector<int>> components = findComponents(workMatrix);
295         cout << "Найдено компонент: " << components.size() << endl;
296         for (size_t i = 0; i < components.size(); i++) {
297             cout << "Компонента " << i + 1 << ": ";
298             for (int v : components[i]) {

```

```

297     }
298     for (int v : components[i]) {
299         cout << v << " ";
300     }
301     cout << endl;
302 }
303
304 // 5. Обходы
305 cout << "\n=== Обходы графа ===" << endl;
306 if (!components.empty() && !components[0].empty()) {
307     int startVertex = components[0][0] - 1; // Берем первую вершину из первой компоненты
308     dfsTraversal(startVertex, workMatrix);
309     bfs(startVertex, workMatrix);
310 }
311
312 // 6. Матрица достижимости
313 cout << "\n=== Матрица достижимости ===" << endl;
314 vector<vector<int>> reachMatrix = reachabilityMatrix(workMatrix);
315 printMatrix(reachMatrix, "Итоговая матрица достижимости");
316
317 // 7. Матрица компонент сильной связности
318 cout << "\n=== Матрица компонент сильной связности ===" << endl;
319 vector<vector<int>> strongCompMatrix = strongComponentsMatrix(reachMatrix);
320 printMatrix(strongCompMatrix, "Матрица компонент сильной связности");
321
322 // Выделение компонент сильной связности
323 vector<bool> vertexProcessed(N, false);
324 cout << "\nКомпоненты сильной связности:\n";
325
326 for (int i = 0; i < N; i++) {
327     if (!vertexProcessed[i]) {
328         cout << "Компонента: ";
329         for (int j = 0; j < N; j++) {
330             if (strongCompMatrix[i][j] == 1) {
331                 cout << j + 1 << " ";
332                 vertexProcessed[j] = true;
333             }
334         }
335         cout << endl;
336     }
337 }
338
339 return 0;
340
341 }

```

4 Тестирование программы

```
Исходная матрица смежности:
:
0 0 0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0 1 0
0 0 0 0 0 0 1 1 0 0
0 0 0 0 0 0 0 1 0 1
0 1 0 0 0 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0
0 0 1 0 0 1 0 0 1 1
0 0 1 1 0 0 0 0 0 0
0 1 0 0 0 0 1 0 0 0
0 0 0 1 0 0 1 0 0 0

Граф неориентированный
Преобразуем в ориентированный (делаем симметричным)...

Ориентированная матрица:
0 0 0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0 1 0
0 0 0 0 0 0 1 1 0 0
0 0 0 0 0 0 0 1 0 1
0 1 0 0 0 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0
0 0 1 0 0 1 0 0 1 1
0 0 1 1 0 0 0 0 0 0
0 1 0 0 0 0 1 0 0 0
0 0 0 1 0 0 1 0 0 0

=== Компоненты связности ===
Найдено компонент: 1
Компонента 1: 1 6 7 3 8 4 10 9 2 5

=== Обходы графа ===
DFS из вершины 1: 1 6 7 3 8 4 10 9 2 5
BFS из вершины 1: 1 6 7 3 9 10 8 2 4 5

=== Матрица достижимости ===
Построение матрицы достижимости:

Шаг 0 (A^1):
0 0 0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0 1 0
0 0 0 0 0 0 1 1 0 0
0 0 0 0 0 0 0 1 0 1
0 1 0 0 0 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0
0 0 1 0 0 1 0 0 1 1
0 0 1 1 0 0 0 0 0 0
```

Шаг 1 (объединение до A^2):

```
1 0 0 0 0 1 1 0 0 0
0 1 0 0 1 0 1 0 1 0
0 0 1 1 0 1 1 1 1 1
0 0 1 1 0 0 1 1 0 1
0 1 0 0 1 0 0 0 1 0
1 0 1 0 0 1 1 0 1 1
1 1 1 1 0 1 1 1 1 1
0 0 1 1 0 0 1 1 0 1
0 1 1 0 1 1 1 0 1 1
0 0 1 1 0 1 1 1 1 1
```

Шаг 2 (объединение до A^3):

```
1 0 1 0 0 1 1 0 1 1
0 1 1 0 1 1 1 0 1 1
1 1 1 1 0 1 1 1 1 1
0 0 1 1 0 1 1 1 1 1
0 1 0 0 1 0 1 0 1 0
1 1 1 1 0 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
0 0 1 1 0 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 0 1 1 1 1 1
```

Шаг 3 (объединение до A^4):

```
1 1 1 1 0 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 0 1 1 1 1 1
0 1 1 0 1 1 1 0 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 0 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
```

Шаг 4 (объединение до A^5):

```
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
```

Шаг 5 (объединение до A^6):

```
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
```

Итоговая матрица достижимости:

```
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
```

=== Матрица компонент сильной связности ===

Матрица компонент сильной связности:

```
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
```

Компоненты сильной связности:

Компонента: 1 2 3 4 5 6 7 8 9 10

5 Вывод

Разработана программа на C++, реализующая полный анализ графа: определение ориентированности, приведение к ориентированному виду, поиск компонент связности, обходы DFS и BFS, построение матриц достижимости и сильной связности.

6 Github

<https://github.com/MAKSPOWERO/math>